

Documentação Completa: Fluxo de Acesso ao Dashboard (Frontend)

Este documento descreve detalhadamente cada arquivo modificado para viabilizar o acesso seguro e dinâmico ao Dashboard no Next.js. O sistema passou de um armazenamento em `localStorage` para **Cookies**, garantindo segurança no lado do servidor e evitando piscadas indesejadas (flickering).

Abaixo está o código **exatamente como está na sua aplicação**, com a explicação linha por linha de como ele funciona.

1. Interceptando a Rota: O Middleware (`middleware.ts`)

O arquivo Middleware fica na raiz do frontend e é executado no servidor **antes** que qualquer página protegida seja aberta no navegador do usuário.

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  // 1. O Next.js tenta encontrar o cookie chamado "token"
  const token = request.cookies.get('token')?.value;

  // 2. Pegamos qual página o usuário está tentando acessar (ex: /dashboard)
  const urlAtual = request.nextUrl.pathname;

  // 3. Se a rota for o dashboard E o usuário não tiver o token...
  if (urlAtual.startsWith('/dashboard') && !token) {
    // Mandamos ele de volta para a tela de login imediatamente!
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // Se a rota for o login E o usuário JÁ TIVER o token...
  if (urlAtual.startsWith('/login') && token) {
    // Mandamos ele direto pro dashboard (afinal, ele já está logado)
    return NextResponse.redirect(new URL('/dashboard', request.url));
  }

  // 4. Se estiver tudo certo, permite que a página carregue normalmente
  return NextResponse.next();
}

// 5. Configuração: Aqui dizemos ao Middleware quais rotas ele deve "vigiar"
export const config = {
  matcher: ['/dashboard/:path*', '/login'],
};
```

Explicação Linha por Linha:

- `import { NextResponse }...`: Importa as ferramentas do Next.js para ler a requisição e forçar redirecionamentos.
- `export function middleware:` A função principal que intercepta o tráfego do usuário.
- `request.cookies.get('token')?.value:` Aqui o servidor extrai o valor do Cookie chamado "token". O `?.value` garante que não quebre se o cookie não existir.
- `const urlAtual = request.nextUrl.pathname:` Descobre qual URL o usuário digitou no navegador (ex: `/dashboard` ou `/login`).
- `if (urlAtual.startsWith('/dashboard') && !token):` Se o usuário tentar invadir o Dashboard sem ter o token, ele é barrado.
- `return NextResponse.redirect(new URL('/login', ...)):` Força o invasor de volta para a tela de login imediatamente.
- `if (urlAtual.startsWith('/login') && token):` Uma regra extra de usabilidade. Se você já está logado e tenta acessar `/login`, ele te empurra para dentro do sistema de novo para você não ter que logar duas vezes.
- `return NextResponse.next():` Se as regras acima não barraram, o guarda abre a catraca e a tela carrega.
- `matcher: ['/dashboard/:path*', '/login']:` Diz ao Next.js para executar essa verificação de segurança apenas nessas duas URLs principais e suas sub-páginas, economizando processamento.

2. A Comunicação com a API (`services/api/authService.ts`)

O serviço responsável por enviar as credenciais para o backend (C#) e lidar com a resposta.

```
export async function cadastrarUsuario(dados: CadastroDados) {
  try {
    const resposta = await fetch(`${API_URL}/api/auth/cadastrar`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(dados),
    });

    if (!resposta.ok) {
      const erroResposta = await resposta.json();

      throw new Error(
        erroResposta.message ||
        "Ocorreu um erro desconhecido ao cadastrar usuário",
      );
    }

    return await resposta.json();
  } catch (err) {
    console.error("Erro na authService:", err);
    throw err;
  }
}
```

Explicação Linha por Linha:

- `fetch(...)`: Cria uma requisição HTTP POST. Note o uso de `${API_URL}/api/auth/cadastrar`, que garante que o endereço esteja perfeitamente alinhado com a porta do C#, evitando erros de "404 Not Found" e quebras na conversão JSON.
- `body: JSON.stringify(dados)`: Transforma os dados preenchidos no formulário num texto JSON que o C# entende.
- `if (!resposta.ok)`: Se o C# devolver um erro (ex: O e-mail já existe, erro 400).
- `const erroResposta = await resposta.json()`: Como a URL foi corrigida, o C# agora envia o erro formatado como JSON, então podemos extrair-lo com segurança.
- `throw new Error(...)`: Dispara o erro extraindo a mensagem (`erroResposta.message`), permitindo que a tela mostre o alerta bonitinho pro usuário.

3. O Formulário de Login (`components/LoginPage/login.tsx`)

Ao receber os dados e enviar pela API, precisamos guardar o Token recebido para o Middleware conseguir trabalhar.

```
const response = await loginUsuario(userLogin);

try {
  if (!response.sucesso) {
    alert("Erro:" + response.message);
    setIsRequest(false);
    return;
  }

  if (response.token && response.role) {
    // Salvamos no Cookie. O 'path=/' diz que o token vale para o site inteiro.
    document.cookie = `token=${response.token}; path=/; max-age=7200`; // 7200s = 2 horas
    document.cookie = `role=${response.role}; path=/; max-age=7200`;
    alert(response.message);
    route.push("/dashboard");
  }
} catch (err) {
  const mensagem =
    err instanceof Error ? err.message : "Erro ao fazer login.";
  alert(mensagem);
}
```

Explicação Linha por Linha:

- `const response = await loginUsuario(userLogin)`: Dispara o login e espera o C# responder.
- `if (!response.sucesso)`: Se a senha estiver errada, mostra um alerta e para a função.
- `if (response.token && response.role)`: Validamos se a API realmente gerou e nos enviou a chave mestra JWT.
- `document.cookie = ...`: Gravamos o biscoito de navegação (cookie).
- `path=/:` Extremamente importante. Diz ao navegador que esse cookie é de acesso global e não apenas da página de `/login`.
- `max-age=7200`: O Cookie tem data de validade de 2 horas (7200 segundos). Após esse tempo, o navegador apaga o cookie sozinho.
- `route.push("/dashboard")`: Como o cookie já está salvo, transferimos o cliente de tela e agora ele passa livremente pelo Middleware (Passo 1).

4. O Header Inteligente (`components/Navbar/navbar.tsx`)

O Navbar precisa esconder os menus nas páginas de login e mostrar o perfil caso logado, evitando erros de layout.

```
// Fora do componente para evitar avisos do linter
const getCookie = (nomeCookie: string) => {
  if (typeof window === "undefined") return null;
  const match = document.cookie.match(new RegExp("(^| )" + nomeCookie + "=(^;|+)$"));
  return match ? match[2] : null;
};

export default function Navbar() {
  const pathname = usePathname();
  const [nomeUsuario, setNomeUsuario] = useState("Cliente");

  useEffect(() => {
    const token = getCookie("token");
    if (token) {
      try {
        const tokenInfo = JSON.parse(atob(token.split(".")[1]));
        const nomeReal = tokenInfo.unique_name || tokenInfo.name || "Cliente";
        const primeiroNome = nomeReal.split(" ")[0];
        // setTimeout resolve aquele erro chato do ESLint sobre cascata de renderização
        setTimeout(() => setNomeUsuario(primeiroNome.charAt(0).toUpperCase() + primeiroNome.slice(1).toLowerCase()), 0);
      } catch (e) {}
    }
  }, []);

  const isDashboard = pathname.startsWith("/dashboard");
}
```

Explicação Linha por Linha:

- `const getCookie = ...`: Função extraída para fora do componente para não acionar alertas (Missing Dependency) do Linter do ESLint.
- `if (typeof window === "undefined")`: O código verifica se está rodando no servidor. Se sim, ignora a leitura de Cookies para não dar erro (o Servidor não possui "document").
- `const [nomeUsuario] = useState("Cliente")`: Valor inicial estático para evitar a quebra do React (Hydration Mismatch) durante a renderização no servidor.
- `useEffect(...)`: Assim que o navegador do usuário termina de carregar a tela inicial, essa função entra em ação.
- `atob(token.split(".")[1])`: O Token JWT é formado por 3 pedaços separados por ponto (.). A parte do meio [1] contém a "Carga Útil" (nome, email). O `atob()` decodifica o texto em Base64, revelando o conteúdo sem a necessidade de instalar bibliotecas pesadas.
- `setTimeout(..., 0)`: Para driblar uma regra rigorosa do Next.js (Cascading Renders), aplicamos um "atraso" de 0ms. Isso tira a execução do fluxo síncrono e resolve os conflitos.
- `charAt(0).toUpperCase()` ...: Tratamento da string para forçar que sempre exiba a primeira letra em maiúscula.
- `const isDashboard`: Verifica via URL se o usuário está ou não no Dashboard.

5. A Saída (Logout) Segura (app/dashboard/page.tsx)

Por fim, quando o usuário está dentro da sala e clica em sair:

```
function handleLogout() {
  // Para apagar um cookie, nós o reescrevemos com uma data de validade no passado (ano 1970)
  document.cookie = "token=; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT";
  document.cookie = "role=; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT";
  router.push("/login");
}
```

Explicação Linha por Linha:

- `function handleLogout`: Invocada no clique do Botão "Sair da Conta".
- `document.cookie = "token=; ..."`: Cookies HTTP não tem um comando simples de "Delete". O modo padrão mundial de excluí-los via frontend é sobrescrevê-los com uma data de validade expirada.
- `expires=Thu, 01 Jan 1970 00:00:00 GMT`: Isso faz o navegador entender que o Cookie "venceu" no dia 1 de Janeiro de 1970, forçando a sua remoção da memória instantaneamente.
- `router.push("/login")`: Uma vez que o Cookie sumiu, levamos o usuário de volta para a tela de login. Como ele não tem mais o Cookie, a catraca do Middleware (Passo 1) volta a trancar as portas do Dashboard.